UNIT: 3 Android User Interface Design

- **User Interface Screen Elements**
  - **Button, EditText, TextView, DatePicker,TimePicker, ProgressBar, ListView, GridView, RadioGroup, ImageButton, Fragement**
- **Designing User Interfaces with Layouts**
  - **Relative Layout, Linear Layout, TableLayout**
- **Dialogs**
- **Drawing and Working with Animation**
  - **Frame By Frame Animation**
  - **Twined Animation**

User Interface Screen Element

- Android UI Controls are those components of Android that are used to design the UI in a more interactive way. It helps us to develop an application that makes user interaction better with the view components. Android provides us a huge range of UI controls of many types such as buttons, text views, etc.
- As we know UI is the only thing that a user interacts with within an Application. This is the reason that we make our Application look aesthetic and, more and more connective. To do so, we need to add the UI controls or we say Input controls in the respective application.

Moving forth we will see some important Android UI controls for our applications:

1. **TextView**
2. **EditText**
3. **Button**
4. **ImageButton**
5. **ToggleButton**
6. **RadioButton**
7. **RadioGroup**
8. **CheckBox**
9. AutoCompleteTextView
10. ProgressBar
11. Spinner
12. TimePicker
13. DatePicker
14. SeekBar
15. AlertDialog
16. Switch
17. RatingBar

1. TextView

- A TextView displays text to the user and optionally allows them to edit it. A TextView is a complete text editor, however the basic class is configured to not allow editing.
- **XML file:**

```
<TextView
        //attributes to describe it
/>
```

Attribute & Description
android:id **This is the ID which uniquely identifies the control.**
android:capitalize **If set, specifies that this TextView has a textual input method and should automatically capitalize what the user types.**
android:fontFamily **Font family (named by string) for the text.**
android:gravity **Specifies how to align the text by the view's x- and/or y-axis when the text is smaller than the view.**
android:hint **Hint text to display when the text is empty.**
android:inputType **The type of data being placed in a text field. Phone, Date, Time, Number, Password etc.**
android:password **Whether the characters of the field are displayed as password dots instead of themselves. Possible value either "true" or "false".**
android:phoneNumber **If set, specifies that this TextView has a phone number input method. Possible value either "true" or "false".**
android:text **Text to display.**
android:textAllCaps **Present the text in ALL CAPS. Possible value either "true" or "false".**
android:textColor **Text color. May be a color value, in the form of "#rgb", "#argb", "#rrggbb", or "#aarrggbb".**
android:textColorHighlight **Color of the text selection highlight.**
android:textColorHint **Color of the hint text. May be a color value, in the form of "#rgb", "#argb", "#rrggbb", or "#aarrggbb".**
android:textIsSelectable **Indicates that the content of a non-editable text can be selected. Possible value either "true" or "false".**
android:textSize **Size of the text. Recommended dimension type for text is "sp" for scaled-pixels (example: 15sp).**
android:textStyle **Style (bold, italic, bolditalic) for the text. You can use or more of the following values separated by '|'.**

2. EditText

- A EditText is an overlay over TextView that configures itself to be editable. It is the predefined subclass of TextView that includes rich editing capabilities.**XML file:**

```
<EditText
//attributes
>
```

Attribute & Description
android:autoText **If set, specifies that this TextView has a textual input method and automatically corrects some common spelling errors.**
android:drawableBottom **This is the drawable to be drawn below the text.**
android:drawableRight **This is the drawable to be drawn to the right of the text.**
android:editable **If set, specifies that this TextView has an input method.**
android:text **This is the Text to display.**

3. Button

- A Button is a Push-button which can be pressed, or clicked, by the user to perform an action.XML file:

```
<Button
//attributes
/>
```

Attribute & Description
android:autoText **If set, specifies that this TextView has a textual input method and automatically corrects some common spelling errors.**
android:drawableBottom **This is the drawable to be drawn below the text.**
android:drawableRight **This is the drawable to be drawn to the right of the text.**
android:editable **If set, specifies that this TextView has an input method.**
android:text **This is the Text to display.**
android:background **This is a drawable to use as the background.**
android:contentDescription **This defines text that briefly describes content of the view.**
android:id **This supplies an identifier name for this view.**
android:onClick **This is the name of the method in this View's context to invoke when the view is clicked.**
android:visibility **This controls the initial visibility of the view.**

4. ImageButton

- An ImageButton is an AbsoluteLayout which enables you to specify the exact location of its children. This shows a button with an image (instead of text) that can be pressed or clicked by the user.**XML file:**

```
<ImageButton
//other attributes...
android:src= “@drawable/add_icon”/>
```

Attribute & Description
android:adjustViewBounds **Set this to true if you want the ImageView to adjust its bounds to preserve the aspect ratio of its drawable.**
android:baseline **This is the offset of the baseline within this view.**
android:baselineAlignBottom **If true, the image view will be baseline aligned with based on its bottom edge.**
android:cropToPadding **If true, the image will be cropped to fit within its padding.**
android:src **This sets a drawable as the content of this ImageView.**

5. ToggleButton

- The toggle button displays the ON/OFF states of a button with a light indicator. A ToggleButton displays checked/unchecked states as a button. It is basically an on/off button with a light indicator.

XML file:

```
<ToggleButton
    //attributes
    android:checked="true"
    android:textOff="OFF"
    android:textOn="ON"/>
```

Attribute & Description
android:disabledAlpha **This is the alpha to apply to the indicator when disabled.**
android:textOff **This is the text for the button when it is not checked.**
android:textOn **This is the text for the button when it is checked.**
android:background **This is a drawable to use as the background.**
android:autoText **If set, specifies that this TextView has a textual input method and automatically corrects some common spelling errors.**
android:drawableBottom **This is the drawable to be drawn below the text.**
android:drawableRight **This is the drawable to be drawn to the right of the text.**
android:editable **If set, specifies that this TextView has an input method.**
android:text **This is the Text to display.**

6. RadioButton

- Radio button in Android is the one that has only two possible states, that are either checked or unchecked. Initially, it is in the unchecked state, once it's checked it can't be unchecked. A RadioButton has two states: either checked or unchecked.This allows the user to select one option from a set.

XML file:

```
<RadioButton
android:text="Radio button"
android:checked="true"/>
```

7. RadioGroup

- It's a group of Radio buttons that are alike. In this, only one of all the buttons can be chosen. A RadioGroup class is used for set of radio buttons.
- If we check one radio button that belongs to a radio group, it automatically unchecks any previously checked radio button within the same group.

XML file:

```
<RadioGroup android:orientation="vertical">
  <RadioButton android:text="Radio Button 1"/>
  <RadioButton android:text="Radio Button 2"/>
  <RadioButton android:text="Radio Button 3"/>
</RadioGroup>
```

Attribute	Description
android:checkedButton	This is the id of child radio button that should be checked by default within this radio group.

8. CheckBox

- A CheckBox is the UI control that has two states that are either checked or unchecked. If we have a group of CheckBox, we can select as many as we want, unlike RadioGroup.
- A CheckBox is an on/off switch that can be toggled by the user. You should use check-boxes when presenting users with a group of selectable options that are not mutually exclusive.

XML file:

```
<CheckBox
android:checked="true"
android:text="CheckBox"
// other attributes
/>
```

9. ProgressBar

- In Android, we have a progress bar that shows the progress of some action that is happening like pasting a file to some location. A progress bar can be in two modes:
- **Determinate Mode:**In this, the progress is shown with the percent of action completed. Also, the time to be taken is already determined.
- **Indeterminate Mode:**In this, there is no idea of when the task would be completed, therefore, it functions continuously.

XML file:

```
<ProgressBar
// attributes, here we define the speed, layout, id, etc.
/>
```

- Progress bars are used to show progress of a task. For example, when you are uploading or downloading something from the internet, it is better to show the progress of download/upload to the user.
- In android there is a class called ProgressDialog that allows you to create progress bar. In order to do this, you need to instantiate an object of this class. Its syntax is.

```
progress.setMessage("Downloading Music :) ");
progress.setProgressStyle(ProgressDialog.STYLE_HORIZONTAL);
progress.setIndeterminate(true);
```

Title & description
getMax() **This method returns the maximum value of the progress.**
incrementProgressBy(int diff) **This method increments the progress bar by the difference of value passed as a parameter.**
setIndeterminate(boolean indeterminate) **This method sets the progress indicator as determinate or indeterminate.**
setMax(int max) **This method sets the maximum value of the progress dialog.**
setProgress(int value) **This method is used to update the progress dialog with some specific value.**
show(Context context, CharSequence title, CharSequence message) **This is a static method, used to display progress dialog.**

The above were some of the very important UI controls, now we will also read about some more UI controls :

1. Spinner

- The Spinner in Android is a User Interface that is used to select a particular option from a list of given options. Spinner in Android is the same as dropdown in HTML. It provides us with a faster way to select an option of our choice. When we click on the down arrow, it shows a list of values from which we can select one. By default, the first value would be shown as the currently selected Value.

2. TimePicker

- Time picker is a UI component that works as an intermediate to select a time of the day. The time chosen by it is shown either in 24 hrs format or in 12hrs format using AM and PM.
- It gives a virtual Clock/watch to select it. This virtual clock makes it easy to choose the time.

3. DatePicker

- Like we have time picker, we have a date picker as UI control too. In this, the System shows a virtual calendar to the users to choose the day.
- This enables the user to choose a particular date using either a calendar or a dropdown. These both are made to make it easier for the user to pick up a date and a time.

4. SeekBar

- In Android, Seekbar is an extended Progress bar. A seekbar comes with a pointer that is draggable throughout the bar either in left or right. This pointer helps to set the progress as well. This helps the user to choose a particular range of values.

5. RatingBar

- A rating bar in Android is an extended version of a seekbar. It is used to give the rating by touching it. In the rating bar, a user can rate at a scale of 5 with a difference of 0.5.
- Its rating is in Stars. The user needs to tap/click the stars.

6. AlertDialog

- Alert Dialog Box is a UI that gives the users an Alert or Warning of something. It appears on the screen in a small window. Once it comes, the user needs to decide or choose an option that it shows.
- For example, when you enter the wrong password for email id.

7. Switch

- In Android, a switch is a two-state UI element that holds either ON or OFF state. ON generally means Yes and OFF means No. By default, a switch is in the OFF state. A user can change its state many times.

8. AutoCompleteTextView

- AutoCompleteTextView is an extension of EditText. In this UI element, the user is provided with a few suggestions of some values/texts. The value can be chosen by the user while filling AutoCompleteTextView.

UI Layouts

- The basic building block for user interface is a View object which is created from the View class and occupies a rectangular area on the screen and is responsible for drawing and event handling. View is the base class for widgets(UI controls), which are used to create interactive UI components like buttons, text fields, etc.
- Types of Layouts
- **LinearLayout**
- **RelativeLayout**
- FrameLayout
- **GridView**
- **ListView**
- **Table Layout**
- Absolute Layout
- ConstraintLayout

LinearLayout:

- This is the most basic layout in android studio, that aligns all the children sequentially either in a horizontal manner or a vertical manner by specifying the android:orientation attribute.
- If one applies android:orientation="vertical" then elements will be arranged one after another in a vertical manner and If you apply android:orientation="horizontal" then elements will be arranged one after another in a horizontal manner.

Attribute & Description
android:id **This is the ID which uniquely identifies the layout.**
android:baselineAligned **This must be a boolean value, either "true" or "false" and prevents the layout from aligning its children's baselines.**
android:divider **This is drawable to use as a vertical divider between buttons. You use a color value, in the form of "#rgb", "#argb", "#rrggbb", or "#aarrggbb".**
android:gravity **This specifies how an object should position its content, on both the X and Y axes. Possible values are top, bottom, left, right, center, center_vertical, center_horizontal etc.**
android:orientation **This specifies the direction of arrangement and you will use "horizontal" for a row, "vertical" for a column. The default is horizontal.**
android:weightSum **Sum up of child weight**

Relative Layout:

- This is very flexible layout used in android for custom layout designing. It gives us the flexibility to position our component/view based on the relative or sibling component's position. Just because it allows us to position the component anywhere we want so it is considered as most flexible layout. For the same reason Relative layout is the most used layout after the Linear Layout in Android. It allow its child view to position relative to each other or relative to the container or another container.
- In Relative Layout, you can use "above, below, left and right" to arrange the component's position in relation to other component. For example, in the below image you can see content is placed in related to Heading.

Attribute & Description
android:layout_above **Positions the bottom edge of this view above the given anchor view ID and must be a reference to another resource, in the form "@[+][package:]type:name"**
android:layout_alignBottom **Makes the bottom edge of this view match the bottom edge of the given anchor view ID and must be a reference to another resource, in the form "@[+][package:]type:name".**
android:layout_alignLeft **Makes the left edge of this view match the left edge of the given anchor view ID and must be a reference to another resource, in the form "@[+][package:]type:name".**
android:layout_alignParentBottom **If true, makes the bottom edge of this view match the bottom edge of the parent. Must be a boolean value, either "true" or "false".**
android:layout_alignParentEnd **If true, makes the end edge of this view match the end edge of the parent. Must be a boolean value, either "true" or "false".**
android:layout_alignParentLeft **If true, makes the left edge of this view match the left edge of the parent. Must be a boolean value, either "true" or "false".**
android:layout_alignParentRight **If true, makes the right edge of this view match the right edge of the parent. Must be a boolean value, either "true" or "false".**
android:layout_alignParentStart **If true, makes the start edge of this view match the start edge of the parent. Must be a boolean value, either "true" or "false".**
android:layout_alignParentTop **If true, makes the top edge of this view match the top edge of the parent. Must be a boolean value, either "true" or "false".**
android:layout_alignRight **Makes the right edge of this view match the right edge of the given anchor view ID and must be a reference to another resource, in the form "@[+][package:]type:name".**
android:layout_alignStart **Makes the start edge of this view match the start edge of the given anchor view ID and must be a reference to another resource, in the form "@[+][package:]type:name".**
android:layout_alignTop **Makes the top edge of this view match the top edge of the given anchor view ID and must be a reference to another resource, in the form "@[+][package:]type:name".**
android:layout_below **Positions the top edge of this view below the given anchor view ID and must be a reference to another resource, in the form "@[+][package:]type:name".**
android:layout_centerHorizontal **If true, centers this child horizontally within its parent. Must be a boolean value, either "true" or "false".**
android:layout_centerInParent **If true, centers this child horizontally and vertically within its parent. Must be a boolean value, either "true" or "false".**
android:layout_centerVertical **If true, centers this child vertically within its parent. Must be a boolean value, either "true" or "false".**
android:layout_toEndOf **Positions the start edge of this view to the end of the given anchor view ID and must be a reference to another resource, in the form "@[+][package:]type:name".**

android:layout_toLeftOf **Positions the right edge of this view to the left of the given anchor view ID and must be a reference to another resource, in the form "@[+][package:]type:name".**
android:layout_toRightOf **Positions the left edge of this view to the right of the given anchor view ID and must be a reference to another resource, in the form "@[+][package:]type:name".**
android:layout_toStartOf **Positions the end edge of this view to the start of the given anchor view ID and must be a reference to another resource, in the form "@[+][package:]type:name".**

GridView:

- In android GridView is a view group that display items in two dimensional scrolling grid (rows and columns), the grid items are not necessarily predetermined but they are automatically inserted to the layout using a ListAdapter. Users can then select any grid item by clicking on it.
- GridView is widely used in android applications. An example of GridView is your default Gallery, where you have number of images displayed using grid.
- Adapter Is Used To Fill Data In Gridview: To fill the data in a GridView we simply use adapter and grid items are automatically inserted to a GridView using an Adapter which pulls the content from a source such as an arraylist, array or database.

Attribute & Description
android:id **This is the ID which uniquely identifies the layout.**
android:columnWidth **This specifies the fixed width for each column. This could be in px, dp, sp, in, or mm.**
android:gravity **Specifies the gravity within each cell. Possible values are top, bottom, left, right, center, center_vertical, center_horizontal etc.**
android:horizontalSpacing **Defines the default horizontal spacing between columns. This could be in px, dp, sp, in, or mm.**
android:numColumns **Defines how many columns to show. May be an integer value, such as "100" or auto_fit which means display as many columns as possible to fill the available space.**
android:stretchMode **Defines how columns should stretch to fill the available empty space, if any.**
android:verticalSpacing **Defines the default vertical spacing between rows. This could be in px, dp, sp, in, or mm.**

ListView:

- List of scrollable items can be displayed in Android using ListView. It helps you to displaying the data in the form of a scrollable list. Users can then select any list item by clicking on it. ListView is default scrollable so we do not need to use scroll View or anything else with ListView.
- ListView is widely used in android applications. A very common example of ListView is your phone contact book, where you have a list of your contacts displayed in a ListView and if you click on it then user information is displayed.

Attribute & Description
android:id **This is the ID which uniquely identifies the layout.**
android:divider **This is drawable or color to draw between list items.**

android:dividerHeight **This specifies height of the divider. This could be in px, dp, sp, in, or mm.**
android:entries **Specifies the reference to an array resource that will populate the ListView.**
android:footerDividersEnabled **When set to false, the ListView will not draw the divider before each footer view. The default value is true.**
android:headerDividersEnabled **When set to false, the ListView will not draw the divider after each header view. The default value is true.**

TableLayout:

- In Android, Table Layout is used to arrange the group of views into rows and columns. Table Layout containers do not display a border line for their columns, rows or cells. A Table will have as many columns as the row with the most cells.
- Android TableLayout going to be arranged groups of views into rows and columns. You will use the <TableRow> element to build a row in the table. Each row has zero or more cells; each cell can hold one View object.

Attribute & Description
android:id **This is the ID which uniquely identifies the layout.**
android:collapseColumns **This specifies the zero-based index of the columns to collapse. The column indices must be separated by a comma: 1, 2, 5.**
android:shrinkColumns **The zero-based index of the columns to shrink. The column indices must be separated by a comma: 1, 2, 5.**
android:stretchColumns **The zero-based index of the columns to stretch. The column indices must be separated by a comma: 1, 2, 5.**

Dialogs:

- A Dialog is small window that prompts the user to a decision or enter additional information.
- Some times in your application, if you wanted to ask the user about taking a decision between yes or no in response of any particular action taken by the user, by remaining in the same activity and without changing the screen, you can use Alert Dialog.
- In order to make an alert dialog, you need to make an object of AlertDialogBuilder which an inner class of AlertDialog.

Syntax:

```
val alertDialogBuilder = AlertDialog.Builder(this)
```

Method type & description
setIcon(Drawable icon) **This method set the icon of the alert dialog box.**
setCancelable(boolean cancel able) **This method sets the property that the dialog can be cancelled or not**
setMessage(CharSequence message) **This method sets the message to be displayed in the alert dialog**
setMultiChoiceItems(CharSequence[] items, boolean[] checkedItems, DialogInterface.OnMultiChoiceClickListener listener) **This method sets list of items to be displayed in the dialog as the content. The selected option will be notified by the listener**

setOnCancelListener(DialogInterface.OnCancelListener onCancelListener) **This method Sets the callback that will be called if the dialog is cancelled.**
setTitle(CharSequence title) **This method set the title to be appear in the dialog**

Example:

```
val builder = AlertDialog.Builder(this@MainActivity)
   //set title for alert dialog
   builder.setTitle("dialogTitle")
   //set message for alert dialog
   builder.setMessage("dialogMessage")
   builder.setIcon(R.mipmap.ic_launcher_round)

   //performing positive action
   builder.setPositiveButton("Yes")
   { dialogInterface, which ->
      Toast.makeText(applicationContext, "clicked yes",
         Toast.LENGTH_LONG).show()
   }
   //performing cancel action
   builder.setNeutralButton("Cancel") { dialogInterface, which ->
      Toast.makeText(
         applicationContext,
         "clicked cancel\n operation cancel",
         Toast.LENGTH_LONG
      ).show()
   }
   //performing negative action
   builder.setNegativeButton("No") { dialogInterface, which ->
      Toast.makeText(applicationContext, "clicked No", Toast.LENGTH_LONG).show()
   }
   // Create the AlertDialog
   val alertDialog: AlertDialog = builder.create()
   // Set other dialog properties
   alertDialog.setCancelable(false)
   alertDialog.show()
```

Drawing and Working with Animation

Animation is the process of adding a motion effect to any view, image, or text. With the help of an animation, you can add motion or can change the shape of a specific view. Animation in Android is generally used to give your UI a rich look and feel.

Frame By Frame Animation/Animate drawable graphics

- Frame-by-frame animation is where a series of images are shown in succession at quick intervals so that the final effect is that of an object moving or changing.
- In Android, frame-by-frame animation is implemented through the class AnimationDrawable. This class is a Drawable. These objects are commonly used as backgrounds for views. AnimationDrawable, in addition to being a Drawable, can take a list of other Drawable resources (like images) and render them at specified intervals.
- To use this AnimationDrawable class, start with a set of Drawable resources (for example, a set of images) placed in the /res/drawable subdirectory. You will then construct an XML file that defines the AnimationDrawable using a list of these images. This XML file needs to be placed in the /res/drawable/frame_animation.xml subdirectory as well.

Frame_animation.xml

```
<?xml version="1.0" encoding="utf-8"?>
<animation-list xmlns:android="http://schemas.android.com/apk/res/android"
   android:oneshot="false">
   <item
      android:duration="100"
      android:drawable="@mipmap/start_1"/>
   <item
      android:duration="100"
      android:drawable="@mipmap/start_2"/>
   <item
      android:duration="100"
      android:drawable="@mipmap/start_3"/>
</animation-list>
```

MainActivity.kt

```
     startanimation.setOnClickListener {
         val bgImage = imageView.background as AnimationDrawable
         bgImage.start()
      }
      stopanimation.setOnClickListener {
         val bgImage = imageView.background as AnimationDrawable
         bgImage.stop()
      }
```

activity_main.xml

```
<LinearLayout
      android:layout_width="match_parent"
      android:layout_height="match_parent"
      android:orientation="vertical">
      <ImageView
         android:id="@+id/imageView"/>

      <LinearLayout
         android:layout_width="match_parent"
         android:layout_height="match_parent"
         android:orientation="horizontal"
         <Button
            android:id="@+id/startanimation"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Start" />
         <Button
            android:id="@+id/stopanimation"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Stop" />
      </LinearLayout>
   </LinearLayout>
```

Twined Animation/View Animation

- Tween Animation is one kind of animation that we only need to set the start and the end. It will supply the lack between the start and the end. It is why we called tween here.
- We have four kinds tween animation: **Alpha, Rotate, Translate, Scale**.
- Tween animation could be set in both code and xml.
- By AnimationSet, we could mix these animations together.
- **By AnimationUtils.loadAnimation, we could load these animations defined in xml.**
- By Animation.AnimationListener, we could add listner to listen to changes of animations.

XML ATTRIBUTES	DESCRIPTION
android:duration	It is used to specify the duration of animation to run
android:fromAlpha	It is the starting alpha value for the animation, where 1.0 means fully opaque and 0.0 means fully transparent
android:toAlpha	It is the ending alpha value
android:id	Sets unique id of the view
android:fromYDelta	It is the change in Y coordinate to be applied at the start of the animation
android:toYDelta	It is the change in Y coordinate to be applied at the end of the animation
android:startOffset	Delay occur when an animation runs (in milliseconds), once start time is reached.
android:pivotX	It represents the X-axis coordinates to zoom from starting point.
android:pivotY	It represents the Y-axis coordinates to zoom from starting point.
android:fromXScale	Starting X size offset,
android:fromYScale	Starting Y size offset,
android:toXScale	Ending of X size offset
android:toYScale	Ending of Y size offset
android:fromDegrees	Starting angular position, in degrees.
android:toDegrees	Ending angular position, in degrees.
android:interpolator	An interpolator defines the rate of change of an animation

bounce.xml :In this animation the text is bounce like a ball.

```
<?xml version="1.0" encoding="utf-8"?>
<set
   xmlns:android="http://schemas.android.com/apk/res/android"
   android:interpolator="@android:anim/linear_interpolator"
   android:fillAfter="true">
   <translate
      android:fromYDelta="100%"
      android:toYDelta="-20%"
      android:duration="300" />
   <translate
      android:startOffset="500"
      android:fromYDelta="-20%"
      android:toYDelta="10%"
      android:duration="150" />
   <translate
      android:startOffset="1000"
      android:fromYDelta="10%"
      android:toYDelta="0"
      android:duration="100" />
```

```
</set>
```

fade_in.xml

In Fade In animation the text will appear from background.

```
<?xml version="1.0" encoding="utf-8"?>
<set xmlns:android="http://schemas.android.com/apk/res/android"
    android:interpolator="@android:anim/linear_interpolator">
    <alpha
        android:duration="1000"
        android:fromAlpha="0.1"
        android:toAlpha="1.0" />
</set>
```

fade_out.xml

In Fade Out animation the colour of text is faded for a particular amount of time.

```
<?xml version="1.0" encoding="utf-8"?>
<set xmlns:android="http://schemas.android.com/apk/res/android"
    android:interpolator="@android:anim/linear_interpolator">
    <alpha
        android:duration="1000"
        android:fromAlpha="1.0"
        android:toAlpha="0.1" />
</set>
```

rotate.xml

In rotate animation the text is rotated for a particular amount of time.

```
<?xml version="1.0" encoding="utf-8"?>
<rotate xmlns:android="http://schemas.android.com/apk/res/android"
    android:duration="1000"
    android:fromDegrees="0"
    android:interpolator="@android:anim/linear_interpolator"
    android:pivotX="50%"
    android:pivotY="50%"
    android:startOffset="0"
    android:toDegrees="360" />
```

slide_down.xml

In this animation the text will come from top to bottom.

```
<?xml version="1.0" encoding="utf-8"?>
<set xmlns:android="http://schemas.android.com/apk/res/android">
    <translate
        android:duration="1000"
        android:fromYDelta="-100%"
        android:toYDelta="0" />
</set>
```

slide_up.xml

In this animation the text will go from bottom to top.

```
<set xmlns:android="http://schemas.android.com/apk/res/android">
    <translate
        android:duration="1000"
        android:fromYDelta="0"
```

```
        android:toYDelta="-100%" />
</set>
```

zoom_in.xml

In this animation the text will appear bigger for a particular amount of time.

```
<?xml version="1.0" encoding="utf-8"?>
<set xmlns:android="http://schemas.android.com/apk/res/android"
    android:fillAfter="true">
    <scale xmlns:android="http://schemas.android.com/apk/res/android"
        android:duration="1000"
        android:fromXScale="1"
        android:fromYScale="1"
        android:pivotX="50%"
        android:pivotY="50%"
        android:toXScale="1.5"
        android:toYScale="1.5">
    </scale>
</set>
```

zoom_out.xml

In this animation the text will appear smaller for a particular amount of time.

```
<?xml version="1.0" encoding="utf-8"?>
<set xmlns:android="http://schemas.android.com/apk/res/android"
    android:fillAfter="true" >
    <scale
        xmlns:android="http://schemas.android.com/apk/res/android"
        android:duration="1000"
        android:fromXScale="1.0"
        android:fromYScale="1.0"
        android:pivotX="50%"
        android:pivotY="50%"
        android:toXScale="0.5"
        android:toYScale="0.5" >
    </scale>
</set>
```